



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de **HONORIS UNITED UNIVERSITIES**

Conception d'un Robot Mobile Autonome

Résolveur de labyrinthe par suivi de mur

2^{ème} année préparatoire
(2AP)

Année Universitaire : 2025-2026

Réalisé par :

Errami Hatim

EL Musauï Mohammed

Aadiati Fouad

Alae Ddin El Mouden

Encadré par :

- Pr. Omar BOUAZIZI

Membres de jury :

- M. Omar BOUAZIZI

- M. Abderrachid Errezgouny

- M. Smail Aitelasri

Résumé

Le présent projet porte sur la conception et la réalisation d'un robot mobile autonome résolveur de labyrinthe basé sur Arduino. Le système utilise des capteurs à ultrasons pour détecter les murs et les obstacles en temps réel. Un algorithme de suivi de mur est implémenté afin de permettre au robot de parcourir un environnement inconnu et d'atteindre la sortie du labyrinthe de manière autonome. La plateforme Arduino, couplée à des moteurs DC et à un châssis robotique, assure le traitement des données et le contrôle du déplacement. Ce prototype constitue un démonstrateur concret des principes de l'intelligence embarquée et de la navigation autonome en robotique mobile.

Abstract

This project focuses on the design and implementation of an autonomous mobile maze-solving robot based on Arduino. The system uses ultrasonic sensors to detect walls and obstacles in real time. A wall-following algorithm is implemented to enable the robot to navigate an unknown environment and reach the maze exit autonomously. The Arduino platform, combined with DC motors and a robotic chassis, handles data processing and motion control. This prototype serves as a concrete demonstrator of embedded intelligence and autonomous navigation principles in mobile robotics.

Remerciement

Nous tenons à exprimer notre profonde gratitude à Monsieur Bouazizi Omar, notre encadrant, pour son soutien constant et ses conseils éclairés tout au long de ce projet. Sa disponibilité, sa patience et son suivi rigoureux nous ont permis d'avancer avec confiance malgré les difficultés rencontrées. Grâce à ses explications claires et à sa vision méthodique, il a su nous orienter efficacement à chaque étape, tout en nous poussant à faire preuve d'autonomie et de réflexion personnelle. Nous lui sommes très reconnaissants pour le temps qu'il nous a accordé, pour la qualité de ses remarques et pour l'encouragement bienveillant dont il a fait preuve jusqu'à la finalisation de ce travail.

Table des matières

Introduction Générale	4
1 Étude fonctionnelle et conception du robot mobile autonome	5
1.1 Présentation du besoin	6
1.2 Analyse fonctionnelle	6
1.2.1 Fonction globale	6
1.2.2 Fonctions de service	7
1.2.3 Contraintes du système	7
1.3 Architecture fonctionnelle du robot	7
1.3.1 Fonction de perception	7
1.3.2 Fonction de décision	8
1.3.3 Fonction d'action	8
1.3.4 Fonction d'alimentation	8
1.4 Architecture matérielle	8
1.4.1 Carte Arduino Uno	9
1.4.2 Capteurs ultrasoniques HC-SR04	9
1.4.3 Driver moteur L298N	10
1.4.4 Moteurs à courant continu	10
1.4.5 Système d'alimentation	11
1.4.6 Châssis du robot	12
1.5 Estimation Du coût	13
1.6 Conclusion	13
2 Réalisation électronique et simulation	14
2.1 schema du cablage	15
2.2 Simulation sous Proteus	16
2.3 Résultats de la simulation et discussion	17
2.4 Conclusion	18

3	Développement logiciel et commande du robot	19
3.1	Introduction	20
3.2	Organigramme de fonctionnement du robot	20
3.3	Algorithme de navigation	21
3.4	Implémentation sous Arduino IDE	22
3.4.1	Déclaration des broches et paramètres	22
3.4.2	Fonction <code>loop()</code>	23
3.4.3	Fonction <code>mesurerDistance()</code>	23
3.5	Fonctions de déplacement	24
3.5.1	Fonction <code>avancer()</code>	24
3.5.2	Fonction <code>arreterRobot()</code>	25
3.5.3	Fonction <code>tournerDroite()</code>	26
3.5.4	Fonction <code>tournerGauche()</code>	26
3.5.5	Fonction <code>demiTour()</code>	27
3.6	Conclusion	27
4	Réalisation pratique et résultats expérimentaux	28
4.1	Introduction	29
4.2	Présentation du prototype réalisé	29
4.3	Tests effectués	29
4.4	Résultats obtenus	29
4.5	Difficultés rencontrées	30
4.6	Perspectives d'amélioration	30
4.7	Conclusion	31
	Conclusion générale	32
	Bibliographie	33

Table des figures

1.1	Prototype du robot mobile autonome réalisé	8
1.2	Carte Arduino Uno utilisée dans le système	9
1.3	Capteur ultrasonique HC-SR04	10
1.4	Module de commande moteur L298N	10
1.5	Moteur à courant continu utilisé dans le robot	11
1.6	Système d'alimentation utilisé pour le robot mobile autonome.	11
1.7	Châssis du robot mobile autonome	12
2.1	Schéma du cablage du systeme	15
3.1	Organigramme de fonctionnement du robot mobile autonome .	20

Liste des tableaux

1.1	Estimation du coût des composants du robot mobile autonome	13
2.1	Affectation des broches de l'Arduino Uno aux capteurs ultrasoniques et au module L298N	17
3.1	Description des principales étapes de l'algorithme de navigation	21

Introduction Générale

La robotique mobile représente aujourd'hui un axe majeur de l'automatisation et de l'intelligence embarquée. Dans un contexte où l'autonomie des systèmes devient un critère clé, la navigation en environnement inconnu occupe une place de plus en plus importante dans les domaines de l'exploration, de la logistique et de la recherche. L'étude d'un robot résolveur de labyrinthe s'inscrit comme une réponse pédagogique et pertinente à ces enjeux technologiques.

Ce projet vise à concevoir et réaliser un robot mobile capable de parcourir un labyrinthe inconnu de manière autonome, en détectant les obstacles grâce à des capteurs, en prenant des décisions de navigation en temps réel et en optimisant son trajet. L'objectif principal est d'implémenter un algorithme de résolution de chemin efficace, de garantir un déplacement stable et précis du robot, et de proposer une solution compacte, économique et fonctionnelle.

La réalisation de ce système repose sur l'utilisation de la plateforme Arduino couplée à des capteurs ultrason, des moteurs DC et un châssis robotique. Grâce à cette architecture, le système est capable d'acquérir des données de l'environnement, de traiter l'information en temps réel via un microcontrôleur, et d'adapter son comportement pour atteindre la sortie du labyrinthe.

Le robot résolveur de labyrinthe se veut ainsi un démonstrateur concret des principes de l'intelligence embarquée et de l'algorithmique de navigation. Il illustre parfaitement comment l'association de capteurs, de microcontrôleurs et d'algorithmes peut répondre à un problème concret, tout en sensibilisant aux contraintes du temps réel et de l'optimisation énergétique.

Chapitre 1

Étude fonctionnelle et conception du robot mobile autonome

1.1 Présentation du besoin

Avec le développement croissant de la robotique mobile, les systèmes autonomes sont de plus en plus utilisés dans des domaines variés tels que l'exploration, la logistique, la surveillance et l'assistance industrielle. Ces applications nécessitent des robots capables de se déplacer de manière autonome dans des environnements parfois inconnus tout en évitant les obstacles présents sur leur trajectoire.

Dans ce contexte, la résolution de labyrinthe constitue un problème classique de navigation autonome permettant de mettre en œuvre les concepts fondamentaux de perception, de prise de décision et de contrôle du mouvement. Un robot résolveur de labyrinthe doit être capable d'analyser son environnement, de détecter les obstacles et de choisir en temps réel la direction la plus appropriée afin d'atteindre sa destination.

L'objectif de ce projet est donc de concevoir et de réaliser un robot mobile autonome capable d'évoluer dans un labyrinthe sans intervention humaine. Le système doit assurer la détection des murs et des obstacles, interpréter les informations acquises par les capteurs et commander les actionneurs afin d'adapter sa trajectoire en fonction de l'environnement rencontré.

Ce projet constitue une application concrète des connaissances acquises en électronique, programmation embarquée et robotique mobile. Il permet également d'illustrer les principes de navigation autonome à travers la réalisation d'un prototype simple, économique et fonctionnel.

1.2 Analyse fonctionnelle

L'analyse des besoins fonctionnels a pour objectif de définir ce que le système doit réaliser, indépendamment des solutions techniques qui seront adoptées par la suite.

1.2.1 Fonction globale

La fonction globale du robot consiste à se déplacer de manière autonome dans un environnement de type labyrinthe afin d'atteindre une sortie sans intervention humaine. Pour accomplir cette mission, le système doit être capable de détecter les obstacles, d'analyser les informations acquises et d'adapter sa trajectoire en temps réel.

1.2.2 Fonctions de service

Afin de satisfaire sa fonction globale, le robot doit assurer les fonctions de service suivantes :

- **Percevoir** : détecter les murs et les obstacles présents dans l'environnement.
- **Traiter** : analyser les données fournies par les capteurs.
- **Décider** : déterminer la trajectoire à suivre en fonction de l'environnement détecté.
- **Agir** : commander les moteurs afin d'assurer le déplacement du robot.
- **S'alimenter** : fournir l'énergie nécessaire au fonctionnement du système.

1.2.3 Contraintes du système

Le robot doit respecter plusieurs contraintes liées à sa conception et à son fonctionnement :

- **Encombrement** : dimensions adaptées aux couloirs du labyrinthe.
- **Coût** : utilisation de composants simples et économiques.
- **Autonomie** : alimentation suffisante pour assurer les phases de fonctionnement.
- **Réactivité** : prise de décision rapide face aux obstacles détectés.
- **Fiabilité** : fonctionnement stable et cohérent dans différentes conditions d'utilisation.

1.3 Architecture fonctionnelle du robot

L'architecture fonctionnelle représente l'organisation générale du robot en plusieurs fonctions complémentaires assurant sa navigation autonome. Chaque fonction joue un rôle spécifique dans la chaîne de traitement de l'information, depuis l'acquisition des données de l'environnement jusqu'à l'exécution des mouvements du robot.

La figure suivante présente l'architecture fonctionnelle du système.

1.3.1 Fonction de perception

La fonction de perception assure l'acquisition des informations provenant de l'environnement extérieur. Elle permet au robot de détecter la présence des murs et des obstacles situés autour de lui et de fournir les données nécessaires à la navigation autonome. Les informations collectées constituent la base du processus de prise de décision.

1.3.2 Fonction de décision

La fonction de décision a pour rôle d'analyser les informations issues de la fonction de perception afin de déterminer l'action la plus appropriée. À partir des données acquises, le système évalue l'état de l'environnement et choisit la trajectoire à suivre pour poursuivre la navigation tout en évitant les obstacles.

1.3.3 Fonction d'action

La fonction d'action assure l'exécution des décisions prises par le système. Elle transforme les commandes générées par l'unité de traitement en mouvements physiques permettant au robot d'avancer, de tourner ou de modifier sa trajectoire en fonction des conditions rencontrées dans le labyrinthe.

1.3.4 Fonction d'alimentation

La fonction d'alimentation fournit l'énergie nécessaire au fonctionnement de l'ensemble du système. Elle garantit l'alimentation des différents éléments du robot et assure son autonomie énergétique pendant les phases de navigation et d'expérimentation.

1.4 Architecture matérielle

L'architecture matérielle du robot mobile autonome est basée sur une structure modulaire composée de 5 blocs fonctionnels interconnectés. Cette organisation permet de séparer clairement les fonctions de perception, de traitement et d'action.

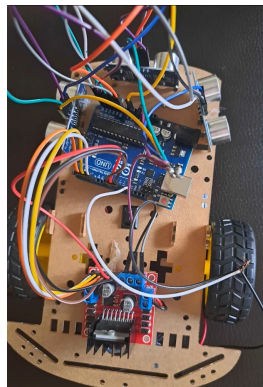


FIGURE 1.1 – Prototype du robot mobile autonome réalisé

1.4.1 Carte Arduino Uno

La carte Arduino Uno constitue l'unité de traitement principale du robot. Basée sur le microcontrôleur ATmega328P, elle assure l'acquisition des données provenant des capteurs, l'exécution de l'algorithme de navigation ainsi que la génération des commandes destinées aux actionneurs.

Grâce à ses entrées et sorties numériques, la carte permet d'interfacer les différents composants du système, notamment les capteurs ultrasoniques et le module de commande des moteurs. Son utilisation est particulièrement adaptée aux projets de robotique en raison de sa simplicité de programmation, de sa fiabilité et de sa large communauté d'utilisateurs.

Dans ce projet, l'Arduino Uno joue le rôle de centre de décision en analysant les informations acquises par les capteurs et en déterminant les actions nécessaires pour assurer le déplacement autonome du robot dans le labyrinthe.



FIGURE 1.2 – Carte Arduino Uno utilisée dans le système

1.4.2 Capteurs ultrasoniques HC-SR04

Les capteurs ultrasoniques HC-SR04 assurent la fonction de perception du robot en permettant la détection des obstacles présents dans son environnement. Ils reposent sur le principe de la mesure du temps de propagation d'une onde ultrasonique entre son émission et la réception de l'écho réfléchi par un obstacle.

Chaque capteur émet une impulsion ultrasonique à une fréquence de 40 kHz puis mesure le temps nécessaire au retour de l'écho. Cette information est ensuite exploitée par la carte Arduino afin de calculer la distance séparant le robot de l'obstacle détecté.

Dans ce projet, trois capteurs HC-SR04 sont utilisés et disposés à l'avant, à gauche et à droite du robot. Cette configuration permet d'obtenir une perception de l'environnement dans plusieurs directions et d'améliorer la prise de décision lors de la navigation dans le labyrinthe.

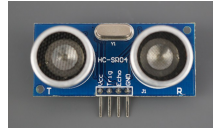


FIGURE 1.3 – Capteur ultrasonique HC-SR04

1.4.3 Driver moteur L298N

Le module L298N est un circuit de commande de moteurs basé sur une architecture en pont en H. Il assure l'interface entre la carte Arduino Uno et les moteurs à courant continu du robot. Son rôle principal est de permettre le contrôle du sens de rotation ainsi que de la vitesse des moteurs à partir des commandes générées par l'unité de traitement.

L'utilisation du driver L298N est nécessaire car les sorties de l'Arduino ne peuvent pas fournir directement le courant requis par les moteurs. Le module reçoit les signaux de commande de la carte Arduino et fournit la puissance nécessaire au fonctionnement des actionneurs.

Dans ce projet, le L298N permet de piloter indépendamment les deux moteurs du robot afin d'assurer les différentes manœuvres de navigation, telles que l'avance, les rotations et les changements de direction.

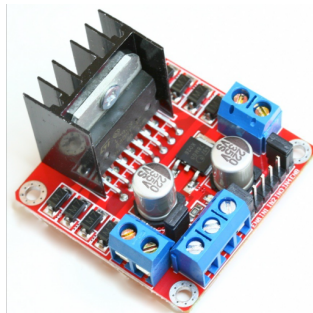


FIGURE 1.4 – Module de commande moteur L298N

1.4.4 Moteurs à courant continu

Les moteurs à courant continu (DC) constituent les actionneurs du robot. Ils assurent la conversion de l'énergie électrique en énergie mécanique afin de produire le mouvement nécessaire au déplacement du système.

Dans ce projet, le robot est équipé de deux moteurs à courant continu montés sur les roues motrices. Leur fonctionnement est commandé par le module L298N, qui permet de contrôler à la fois leur vitesse et leur sens

de rotation. Cette configuration offre au robot la possibilité d'effectuer différentes manœuvres, telles que l'avance, la rotation à gauche, la rotation à droite et le demi-tour.

Le choix des moteurs DC se justifie par leur simplicité d'utilisation, leur faible coût et leur compatibilité avec les applications de robotique mobile éducative. Ils constituent une solution adaptée aux besoins du projet en assurant un déplacement fiable du robot dans l'environnement du labyrinthe.



FIGURE 1.5 – Moteur à courant continu utilisé dans le robot

1.4.5 Système d'alimentation

Le système d'alimentation fournit l'énergie nécessaire au fonctionnement de l'ensemble des composants du robot. Il assure l'alimentation de la carte Arduino Uno, des capteurs ultrasoniques HC-SR04 ainsi que des moteurs à courant continu via le module de commande L298N.

Dans ce projet, l'alimentation est assurée par deux batteries rechargeables Li-ion de 3,7 V montées en série, permettant d'obtenir une tension totale de 7,4 V. Cette configuration offre une autonomie adaptée aux besoins du prototype tout en garantissant une alimentation stable des différents composants du système.

Le choix des batteries Li-ion se justifie par leur faible encombrement, leur légèreté et leur bonne densité énergétique, ce qui les rend particulièrement adaptées aux applications de robotique mobile.



FIGURE 1.6 – Système d'alimentation utilisé pour le robot mobile autonome.

1.4.6 Châssis du robot

Le châssis constitue la structure mécanique du robot. Il assure le support et la fixation de l'ensemble des composants électroniques et mécaniques du système, notamment la carte Arduino Uno, les capteurs ultrasoniques, le module L298N, les moteurs ainsi que le système d'alimentation.

Dans ce projet, un châssis à deux roues motrices a été utilisé en raison de sa simplicité de montage et de sa bonne stabilité. Sa conception permet une intégration efficace des différents composants tout en garantissant la mobilité du robot dans les couloirs du labyrinthe.

Le choix de ce châssis se justifie également par sa légèreté, sa robustesse et son adaptation aux applications de robotique mobile éducative. Il constitue ainsi une base mécanique adéquate pour la réalisation du prototype.



FIGURE 1.7 – Châssis du robot mobile autonome

1.5 Estimation Du coût

Composant	Description technique	Prix (Dhs)	Qté	Coût
Arduino Uno	Carte microcontrôleur AT-mega328P	140	1	140
Driver L298N	Double pont en H	40	1	40
Capteur HC-SR04	Capteur ultrasonique	30	3	90
Câbles Dupont M-F	Connexions électriques	20	1	20
Châssis 2WD	Support mécanique du robot	40	1	40
Batteries Li-ion 3,7 V	Source d'alimentation	70	1	70
Moteurs DC avec roues	Actionneurs de déplacement	40	1	40
Total				440

TABLE 1.1 – Estimation du coût des composants du robot mobile autonome

1.6 Conclusion

Dans ce chapitre, nous avons présenté l'étude fonctionnelle ainsi que l'architecture générale du robot mobile autonome. L'analyse des besoins a permis d'identifier les principales fonctions que doit assurer le système, notamment la perception de l'environnement, la prise de décision, le déplacement et l'alimentation énergétique.

L'étude de l'architecture fonctionnelle et matérielle a également permis de justifier le choix des différents composants utilisés dans la réalisation du prototype. L'association de la carte Arduino Uno, des capteurs ultrasoniques HC-SR04, du module L298N, des moteurs à courant continu et du système d'alimentation constitue une solution adaptée aux objectifs du projet.

Cette phase de conception constitue une base essentielle pour la réalisation du robot. Le chapitre suivant sera consacré à la mise en œuvre matérielle et logicielle du système, ainsi qu'à la programmation de l'algorithme de navigation autonome.

Chapitre 2

Réalisation électronique et simulation

La réalisation électronique constitue l'étape de concrétisation du robot résolveur de labyrinthe. À partir du cahier des charges fonctionnel, cette phase consiste à sélectionner et à interconnecter les différents composants afin d'obtenir un système cohérent et opérationnel.

Dans un premier temps, nous présentons le schéma de câblage réalisé sous Proteus ISIS, illustrant l'interconnexion de la carte Arduino Uno, des trois capteurs ultrasoniques HC-SR04, du driver moteur L298N et des moteurs à courant continu. Dans un second temps, les différents sous-systèmes sont analysés, notamment l'acquisition des distances, le traitement des informations et la commande des moteurs. Cette démarche permet de valider l'architecture du système avant sa mise en œuvre pratique.

2.1 schéma du câblage

Ce schéma ci - dessus présente le câblage électronique du robot suiveur de labyrinthe. L'Arduino Uno constitue le cerveau du système et centralise les signaux des 3 capteurs ultrason HC-SR04 pour détecter les obstacles. Le module L298N reçoit les commandes de l'Arduino pour piloter les deux moteurs CC, tandis que la batterie 7.4V alimente l'ensemble du montage.

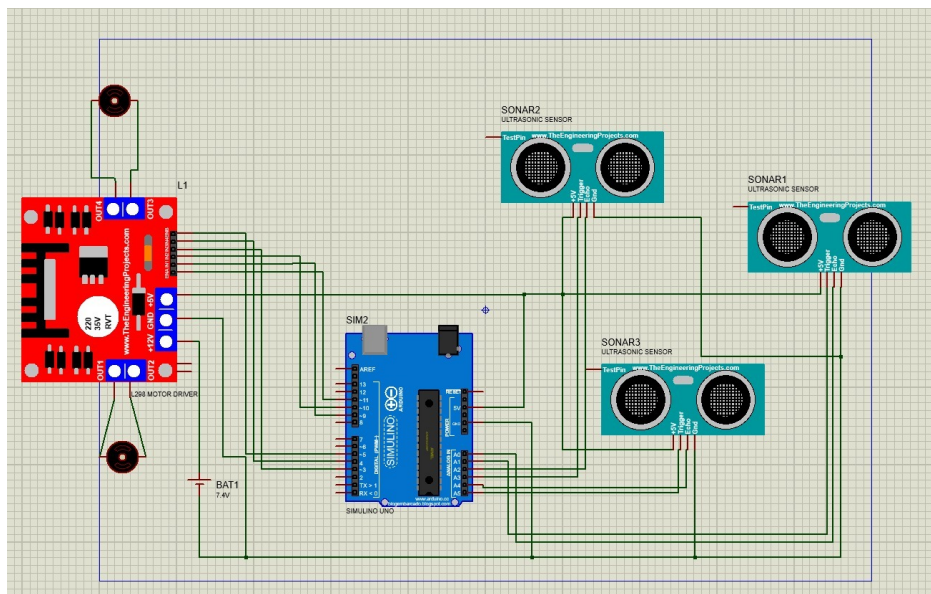


FIGURE 2.1 – Schéma du câblage du système

2.2 Simulation sous Proteus

Avant le montage réel du robot, une simulation du circuit a été effectuée sous Proteus afin de vérifier le câblage et la cohérence du programme. Cette phase permet de détecter les erreurs de connexion et de valider la logique d'évitement sans endommager les composants.

L'architecture simulée est basée sur les éléments suivants :

- **Arduino Uno R3** : unité de traitement qui récupère les distances et commande la locomotion.
- **3 capteurs HC-SR04** nommés SONAR1, SONAR2, SONAR3 : placés respectivement à droite, à l'avant et à gauche. Chaque capteur est alimenté en 5V/GND et ses broches Trig/Echo sont reliées aux broches digitales de l'Arduino.
- **Module L298N** : driver connecté aux broches PWM de l'Arduino pour contrôler la vitesse et le sens de rotation.
- **2 moteurs CC** : Moteur A connecté sur OUT1/OUT2 du L298N, Moteur B sur OUT3/OUT4. Ils assurent l'avancement et les rotations du robot.
- **Batterie 7.4V** : connectée sur l'entrée +12V/GND du L298N. Le régulateur 5V du L298N alimente l'Arduino via la broche 5V.

Lors de la simulation, l'Arduino mesure en permanence les distances fournies par les trois capteurs ultrasoniques. Lorsque la distance détectée à l'avant devient inférieure au seuil fixé à 15 cm, le robot analyse les mesures latérales afin de choisir une nouvelle direction. Il privilégie d'abord une rotation à droite, puis à gauche si nécessaire, et effectue un demi-tour lorsque toutes les directions sont bloquées.

Broche Arduino	Fonction	Connecté à
A0	TRIG capteur avant	HC-SR04 Avant
A1	ECHO capteur avant	HC-SR04 Avant
A2	TRIG capteur gauche	HC-SR04 Gauche
A3	ECHO capteur gauche	HC-SR04 Gauche
A4	TRIG capteur droit	HC-SR04 Droit
A5	ECHO capteur droit	HC-SR04 Droit
D11 (PWM)	ENA	Driver L298N (moteur droit)
D9	IN1	Driver L298N
D10	IN2	Driver L298N
D5 (PWM)	ENB	Driver L298N (moteur gauche)
D3	IN3	Driver L298N
D4	IN4	Driver L298N
5V	Alimentation	Capteurs HC-SR04
GND	Masse commune	L298N et capteurs

TABLE 2.1 – Affectation des broches de l’Arduino Uno aux capteurs ultrasoniques et au module L298N

2.3 Résultats de la simulation et discussion

La simulation réalisée sous Proteus a permis de vérifier la cohérence du schéma électronique ainsi que le fonctionnement général de l’algorithme de navigation. Les différents composants du système, notamment la carte Arduino Uno, les capteurs ultrasoniques HC-SR04 et le driver moteur L298N, ont correctement interagi dans l’environnement de simulation.

Les résultats obtenus ont montré que les capteurs étaient capables de détecter les obstacles et que l’algorithme prenait les décisions attendues en fonction des distances mesurées. Les commandes de déplacement (avance, rotation à gauche, rotation à droite et demi-tour) ont également pu être validées au niveau logique.

Cependant, les performances observées en simulation n’ont pas été totalement reproduites lors de la réalisation pratique du prototype. Des écarts entre le comportement simulé et le comportement réel ont été constatés, ce qui met en évidence les limites des simulations face aux contraintes physiques telles que les variations des moteurs, les perturbations des capteurs et les conditions réelles de fonctionnement.

Malgré ces difficultés, la phase de simulation a constitué une étape importante du projet en permettant de valider l’architecture électronique et de réduire les erreurs de conception avant le passage à la réalisation matérielle.

2.4 Conclusion

Dans ce chapitre, nous avons présenté la réalisation électronique du robot ainsi que la simulation de son fonctionnement sous Proteus. Le schéma de câblage a permis de définir les interconnexions entre la carte Arduino Uno, les capteurs ultrasoniques HC-SR04, le driver moteur L298N et les moteurs DC.

La phase de simulation a constitué une étape importante dans la validation du système en permettant de vérifier la cohérence de l'architecture électronique et le fonctionnement général de l'algorithme de navigation. Elle a également permis d'identifier certaines contraintes liées à l'intégration des différents composants.

Bien que les résultats obtenus en simulation aient été satisfaisants, le passage à la réalisation pratique a révélé plusieurs difficultés qui n'ont pas pu être entièrement résolues durant le projet. Néanmoins, cette étape a permis d'acquérir une meilleure compréhension du fonctionnement global du système et a servi de base pour le développement du logiciel embarqué présenté dans le chapitre suivant.

Chapitre 3

Développement logiciel et commande du robot

3.1 Introduction

Après la conception du robot et la validation de son architecture électronique par simulation, l'étape suivante consiste à développer la partie logicielle assurant son fonctionnement autonome. Cette phase a pour objectif de concevoir et d'implémenter l'algorithme de navigation permettant au robot de se déplacer dans un environnement inconnu tout en évitant les obstacles.

Le programme a été développé sous l'environnement Arduino IDE et repose sur l'exploitation des informations fournies par les capteurs ultrasoniques HC-SR04. À partir de ces données, le robot prend des décisions en temps réel afin de choisir la direction la plus appropriée selon la stratégie de navigation adoptée.

Dans ce chapitre, nous présentons l'organigramme de fonctionnement du système, l'algorithme de navigation utilisé ainsi que les principales fonctions développées pour assurer le déplacement autonome du robot.

3.2 Organigramme de fonctionnement du robot

Afin de représenter le fonctionnement de l'algorithme de navigation, un organigramme a été élaboré. Celui-ci décrit les différentes décisions prises par le robot en fonction des informations fournies par les capteurs ultrasoniques. Le principe adopté repose sur la règle de la main droite, permettant au robot de se déplacer de manière autonome tout en évitant les obstacles présents dans son environnement.

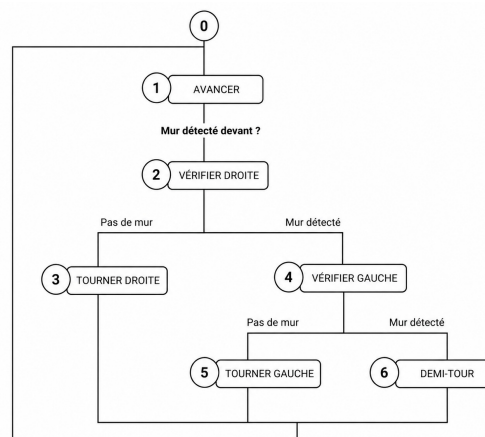


FIGURE 3.1 – Organigramme de fonctionnement du robot mobile autonome

Étape	Description
0	Initialisation de la carte Arduino, des capteurs ultrasoniques et du module de commande moteur.
1	Le robot avance tant qu'aucun obstacle n'est détecté devant lui.
2	En présence d'un obstacle à l'avant, le robot vérifie si le passage à droite est libre.
3	Si le passage à droite est libre, le robot effectue une rotation vers la droite.
4	Si le passage à droite est bloqué, le robot vérifie la disponibilité du passage à gauche.
5	Si le passage à gauche est libre, le robot effectue une rotation vers la gauche.
6	Si tous les passages sont bloqués, le robot réalise un demi-tour avant de reprendre sa navigation.

TABLE 3.1 – Description des principales étapes de l'algorithme de navigation

3.3 Algorithme de navigation

L'algorithme de navigation constitue l'élément central du fonctionnement du robot mobile autonome. Son rôle est de permettre au robot de se déplacer dans un environnement inconnu tout en évitant les obstacles détectés par les capteurs ultrasoniques.

La stratégie adoptée repose sur la règle de la main droite. Cette méthode consiste à privilégier le passage situé à droite lorsque plusieurs directions sont possibles. Le robot analyse en permanence les distances mesurées par les trois capteurs ultrasoniques placés à l'avant, à gauche et à droite du châssis.

Le principe de fonctionnement de l'algorithme peut être résumé comme suit :

- Si aucun obstacle n'est détecté à l'avant, le robot continue d'avancer.
- Lorsqu'un obstacle est détecté devant le robot, celui-ci vérifie en priorité si le passage à droite est libre.
- Si la distance mesurée à droite est supérieure au seuil fixé, le robot effectue une rotation vers la droite.
- Si le passage à droite est bloqué, le robot analyse la disponibilité du passage à gauche.
- Si le passage à gauche est libre, une rotation vers la gauche est réalisée.
- Si les trois directions sont obstruées, le robot effectue un demi-tour avant de reprendre sa navigation.

Cette approche présente l'avantage d'être simple à implémenter tout en

offrant un comportement autonome satisfaisant pour l'exploration et la résolution de labyrinthes simples.

3.4 Implémentation sous Arduino IDE

L'algorithme de navigation a été implémenté à l'aide de l'environnement Arduino IDE. Cette étape consiste à programmer la carte Arduino Uno afin d'assurer l'acquisition des données des capteurs ultrasoniques, le traitement des informations recueillies et la commande des moteurs. Le programme développé permet ainsi au robot d'exécuter de manière autonome les différentes actions nécessaires à sa navigation.

3.4.1 Déclaration des broches et paramètres

Cette partie du programme permet de définir les broches utilisées pour l'interfaçage des capteurs ultrasoniques et du module de commande moteur L298N. Les paramètres de fonctionnement du robot, tels que la vitesse des moteurs et la distance minimale de détection des obstacles, sont également initialisés.

```
int ENA = 11;
int IN1 = 9;
int IN2 = 10;

int ENB = 5;
int IN3 = 3;
int IN4 = 4;

int TRG_AV = A0;
int ECHO_AV = A1

int TRIG_G = A2
int ECHO_G = A3

int TRIG_D = A4
int ECHO_D = A5

int distanceMur = 15;
```

Listing 3.1 – Déclaration des broches et paramètres

3.4.2 Fonction loop()

La fonction `loop()` constitue la boucle principale du programme. Exécutée de manière continue, elle assure la lecture des distances mesurées par les capteurs ultrasoniques et applique l'algorithme de navigation du robot. En fonction des informations recueillies, le système décide d'avancer, de tourner à droite, de tourner à gauche ou d'effectuer un demi-tour afin d'éviter les obstacles rencontrés.

```
void loop() {

    long distAvant  = mesurerDistance(TRG_AV, ECHO_AV);
    long distGauche = mesurerDistance(TRIG_G, ECHO_G);
    long distDroite = mesurerDistance(TRIG_D, ECHO_D);

    if (distAvant > distanceMur) {
        avancer();
    }
    else {
        arreterRobot();

        if (distDroite > distanceMur) {
            tournerDroite();
        }
        else if (distGauche > distanceMur) {
            tournerGauche();
        }
        else {
            demiTour();
        }
    }
}
```

Listing 3.2 – Boucle principale de navigation

3.4.3 Fonction mesurerDistance()

La fonction `mesurerDistance()` permet de mesurer la distance séparant le robot d'un obstacle à l'aide d'un capteur. Son principe de fonctionnement repose sur l'émission d'une onde ultrasonique puis sur la mesure du temps nécessaire à son retour après réflexion sur un obstacle. La distance est ensuite calculée à partir de la vitesse de propagation du son dans l'air.

```
long mesurerDistance(int trigPin, int echoPin) {
```

```

digitalWrite(trigPin, LOW);
delayMicroseconds(2);

digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

long duree = pulseIn(echoPin, HIGH);

long distance = duree * 0.034 / 2;

return distance;
}

```

Listing 3.3 – Fonction de mesure de distance

Cette fonction est utilisée par les trois capteurs ultrasoniques du robot afin d’obtenir les distances mesurées à l’avant, à gauche et à droite. Ces informations constituent les données d’entrée de l’algorithme de navigation et permettent au robot de prendre les décisions appropriées en temps réel.

3.5 Fonctions de déplacement

Les fonctions de déplacement assurent le contrôle des moteurs du robot à travers le module de commande L298N. Elles permettent d’exécuter les différentes manœuvres nécessaires à la navigation autonome, notamment l’avance, l’arrêt, les rotations à droite et à gauche ainsi que le demi-tour. Ces fonctions sont appelées par l’algorithme de navigation en fonction des informations fournies par les capteurs ultrasoniques.

3.5.1 Fonction avancer()

La fonction `avancer()` assure le déplacement du robot en ligne droite. Afin de maintenir une distance raisonnable par rapport aux murs du labyrinthe, une correction de trajectoire est appliquée à partir des mesures fournies par les capteurs latéraux. Lorsque le robot se rapproche excessivement d’un mur, la vitesse des moteurs est ajustée afin de réorienter progressivement sa trajectoire vers le centre du couloir.

```

void avancer(int distanceGauche, int distanceDroite) {

    int vitesseDroite = 175;

```

```

int vitesseGauche = 190;

if (distanceGauche < 5) {
    vitesseDroite += 10;
    vitesseGauche -= 10;
}

if (distanceDroite < 5) {
    vitesseDroite -= 10;
    vitesseGauche += 10;
}

analogWrite(ENA, vitesseDroite);
analogWrite(ENB, vitesseGauche);

digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);

digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);
}

```

Listing 3.4 – Fonction avancer()

3.5.2 Fonction arreterRobot()

Cette fonction assure l'arrêt immédiat du robot en interrompant la commande des moteurs. Elle est principalement utilisée lorsqu'un obstacle est détecté devant le robot avant la prise d'une nouvelle décision de déplacement.

```

void arreterRobot() {

    analogWrite(ENA, 0);
    analogWrite(ENB, 0);

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

```

Listing 3.5 – Fonction arreterRobot()

3.5.3 Fonction tournerDroite()

Cette fonction réalise une rotation du robot vers la droite en appliquant des commandes opposées aux moteurs. Elle est exécutée lorsque le passage situé à droite est libre.

```
void tournerDroite() {  
  
    analogWrite(ENA, 180);  
    analogWrite(ENB, 190);  
  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
  
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);  
  
    delay(260);  
}
```

Listing 3.6 – Fonction tournerDroite()

3.5.4 Fonction tournerGauche()

Cette fonction permet au robot d'effectuer une rotation vers la gauche afin de poursuivre sa navigation lorsqu'un passage est disponible dans cette direction.

```
void tournerGauche() {  
  
    analogWrite(ENA, 180);  
    analogWrite(ENB, 190);  
  
    digitalWrite(IN1, HIGH);  
    digitalWrite(IN2, LOW);  
  
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, HIGH);  
  
    delay(260);  
}
```

Listing 3.7 – Fonction tournerGauche()

3.5.5 Fonction demiTour()

La fonction `demiTour()` est utilisée lorsque toutes les directions sont bloquées. Elle commande une rotation de 180° permettant au robot de revenir sur son trajet et de rechercher un nouvel itinéraire.

```
void demiTour() {  
  
    analogWrite(ENA, 180);  
    analogWrite(ENB, 190);  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);  
    delay(520);  
}
```

Listing 3.8 – Fonction `demiTour()`

3.6 Conclusion

Dans ce chapitre, nous avons présenté le développement logiciel du robot mobile autonome. Après l'élaboration de l'organigramme de fonctionnement et la définition de l'algorithme de navigation, l'implémentation a été réalisée sous l'environnement Arduino IDE. Les différentes fonctions développées permettent la mesure des distances, la prise de décision et la commande des moteurs afin d'assurer la navigation autonome du robot.

L'ensemble des éléments logiciels présentés constitue le cœur du système de commande du robot. Le chapitre suivant sera consacré à la réalisation pratique du prototype ainsi qu'à l'analyse des résultats obtenus lors des essais expérimentaux.

Chapitre 4

Réalisation pratique et résultats expérimentaux

4.1 Introduction

Après la conception et le développement du système, une phase de réalisation pratique a été entreprise afin de valider le fonctionnement du robot mobile autonome. Cette étape consiste à assembler les différents composants matériels, à intégrer le programme développé et à réaliser une série d'essais expérimentaux. Les résultats obtenus permettent d'évaluer les performances du système et d'identifier les principales difficultés rencontrées lors de sa mise en œuvre.

4.2 Présentation du prototype réalisé

Le prototype réalisé est constitué d'un châssis mobile équipé de deux moteurs à courant continu, d'une carte Arduino Uno, de trois capteurs ultrasoniques HC-SR04, d'un module de commande moteur L298N ainsi que d'un système d'alimentation basé sur deux batteries Li-ion de 3,7 V. L'ensemble des composants a été assemblé et câblé conformément à l'architecture définie lors de la phase de conception.

Une fois l'intégration matérielle terminée, le programme de navigation a été chargé dans la carte Arduino afin de permettre au robot d'effectuer ses déplacements de manière autonome.

4.3 Tests effectués

Afin d'évaluer le comportement du robot, plusieurs essais ont été réalisés dans un environnement reproduisant les conditions d'un labyrinthe. Ces tests avaient pour objectif de vérifier le fonctionnement des capteurs ultrasoniques, la commande des moteurs ainsi que l'exécution de l'algorithme de navigation.

Les essais ont principalement porté sur la détection des obstacles, les changements de direction et la capacité du robot à suivre les parois du parcours tout en évitant les collisions.

4.4 Résultats obtenus

Les essais réalisés ont permis de valider le fonctionnement individuel des principaux composants du système. Les capteurs ultrasoniques ont été capables de détecter la présence d'obstacles et de transmettre les informations à la carte Arduino. De même, les moteurs ont correctement répondu aux commandes générées par le programme.

Cependant, lors des essais en conditions réelles, le robot n'a pas réussi à assurer une navigation autonome stable dans le labyrinthe. Bien que certaines manœuvres aient été exécutées correctement, le comportement global du système est resté insuffisant pour garantir une résolution complète du parcours.

Ces résultats ont néanmoins permis d'identifier plusieurs limitations techniques et de mieux comprendre les contraintes liées à l'intégration matérielle et logicielle d'un robot mobile autonome.

4.5 Difficultés rencontrées

Au cours de la phase expérimentale, plusieurs difficultés ont été observées. La principale concerne la précision des déplacements du robot. Les différences de vitesse entre les moteurs ont entraîné des écarts de trajectoire, rendant difficile le suivi correct des parois du labyrinthe.

Par ailleurs, les mesures fournies par les capteurs ultrasoniques présentaient parfois des variations pouvant affecter la prise de décision de l'algorithme. Ces imprécisions ont occasionnellement provoqué des changements de direction inadaptés.

Des contraintes mécaniques liées au châssis, à la fixation des composants ainsi qu'à l'alimentation électrique ont également influencé les performances du système. Enfin, certaines différences entre les conditions de simulation et les conditions réelles n'ont pas permis de reproduire exactement les résultats obtenus sous Proteus.

4.6 Perspectives d'amélioration

Afin d'améliorer les performances du robot, plusieurs pistes peuvent être envisagées. Une première amélioration consisterait à réaliser une calibration plus précise des moteurs afin d'obtenir des déplacements plus réguliers.

L'utilisation de capteurs supplémentaires ou plus performants permettrait également d'améliorer la fiabilité de la détection des obstacles. De même, l'intégration d'encodeurs de roues offrirait un meilleur contrôle de la trajectoire du robot.

Sur le plan logiciel, l'optimisation de l'algorithme de navigation ainsi que l'ajout de méthodes de correction plus avancées contribueraient à améliorer la précision des déplacements. Enfin, une optimisation de la structure mécanique et de l'alimentation pourrait accroître la stabilité générale du système.

4.7 Conclusion

Ce chapitre a présenté la réalisation pratique du robot mobile autonome ainsi que les différents essais effectués pour évaluer son fonctionnement. Les tests réalisés ont permis de vérifier l'intégration des composants matériels et logiciels développés au cours du projet.

Bien que les performances obtenues n'aient pas entièrement répondu aux objectifs fixés, cette expérience a permis d'acquérir des connaissances importantes dans les domaines de la robotique mobile, de l'électronique embarquée et de la programmation des systèmes autonomes. Les difficultés rencontrées constituent également une base de réflexion pour de futures améliorations du prototype.

Conclusion générale

Ce projet avait pour objectif la conception et la réalisation d'un robot mobile autonome capable de se déplacer dans un environnement de type labyrinthe en utilisant des capteurs ultrasoniques pour la détection des obstacles. Pour atteindre cet objectif, une étude fonctionnelle a d'abord été menée afin de définir les besoins du système et de choisir les composants matériels adaptés.

La réalisation du projet a nécessité l'intégration de plusieurs éléments, notamment une carte Arduino Uno, des capteurs ultrasoniques HC-SR04, un module de commande moteur L298N, des moteurs à courant continu ainsi qu'un système d'alimentation autonome. Une simulation du système a également été effectuée sous Proteus afin de valider le schéma électronique et le principe de fonctionnement avant la phase de réalisation pratique.

Par la suite, un algorithme de navigation basé sur la règle de la main droite a été développé et implémenté sous Arduino IDE. Cette étape a permis de mettre en œuvre les différentes fonctions de mesure, de prise de décision et de commande des moteurs nécessaires au déplacement autonome du robot.

Les essais réalisés ont permis de valider le fonctionnement de plusieurs composants du système. Toutefois, les résultats obtenus en conditions réelles n'ont pas totalement répondu aux objectifs fixés. Certaines limitations liées à la précision des capteurs, à la calibration des moteurs ainsi qu'aux contraintes mécaniques ont affecté les performances globales du prototype. Malgré ces difficultés, ce projet a constitué une expérience enrichissante permettant d'acquérir des compétences dans les domaines de l'électronique embarquée, de la programmation des systèmes autonomes et de la robotique mobile.

Enfin, plusieurs perspectives d'amélioration peuvent être envisagées, notamment l'intégration d'encodeurs de roues, l'utilisation de capteurs plus performants ainsi que l'optimisation de l'algorithme de navigation. Ces améliorations permettraient d'accroître la précision des déplacements et la fiabilité du système dans des environnements plus complexes.

Bibliographie

- [1] Arduino, *Arduino Uno Rev3 Documentation officielle*. Disponible sur : <https://www.arduino.cc>
- [2] Arduino, *Arduino IDE Documentation*. Disponible sur : <https://docs.arduino.cc>
- [3] STMicroelectronics, *L298N Dual Full-Bridge Driver Datasheet*. Disponible sur : <https://www.st.com>
- [4] Components101, *HC-SR04 Ultrasonic Sensor Technical Specifications*. Disponible sur : <https://components101.com>
- [5] Labcenter Electronics, *Proteus Design Suite User Guide*. Disponible sur : <https://www.labcenter.com>
- [6] Banzi M., Shiloh M., *Getting Started with Arduino*, Maker Media, 2022.
- [7] Siegwart R., Nourbakhsh I., Scaramuzza D., *Introduction to Autonomous Mobile Robots*, MIT Press, 2011.